CS6886W - Systems Engineering for Deep Learning (Assignment 1)

Exploring VGG6 on CIFAR-10 in TensorFlow | Rakesh Kumar (Roll No: CS25M530)

PUBLIC GITHUB REPOSITORY LINK (Question 5 Requirement):
Repository URL : https://github.com/Rakesh4ever/iitm-cs6886w-vgg6-cifar10
Driver Notebook: https://github.com/Rakesh4ever/iitm-cs6886w-vgg6-cifar10/blob/main/driver.ipynb
Modular Package: src/dataset.py, src/model.py, src/train.py, src/evaluate.py, src/sweep.py

Question 1: Training Baseline (10 Points)

Q1(a) Normalization & Data Augmentation Transforms Used (5 pts):

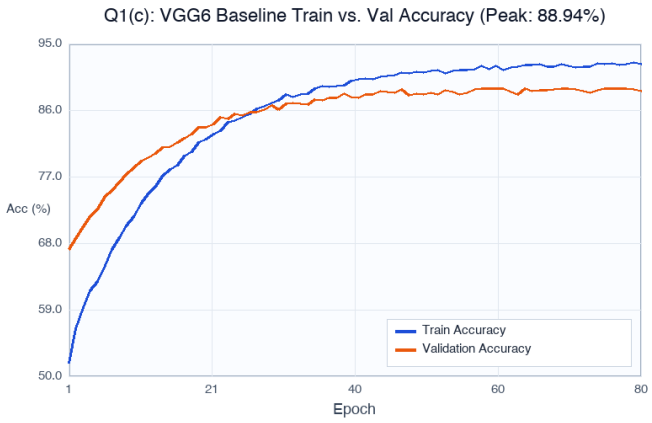
1. Pixel Intensity Scaling: Raw 8-bit RGB images in [0, 255] are cast to tf.float32 and divided by 255.0 to lie in [0.0, 1.0].
2. Channel-Wise Z-Score Standardization: Each RGB channel is standardized using CIFAR-10 training set statistics mean = (0.4914, 0.4822, 0.4465) and std = (0.2470, 0.2435, 0.2616) via x_norm = (x - mean) / std.
3. Dynamic Data Augmentation (Training Set Only - 45,000 images):
   - Reflection padding by 4 pixels on all borders (32x32x3 -> 40x40x3).
   - Random crop back to 32x32x3 (tf.image.random_crop) + Random horizontal flip (p = 0.5).
   - Validation (5,000 images) and Test (10,000 images) pipelines apply only Z-score normalization.

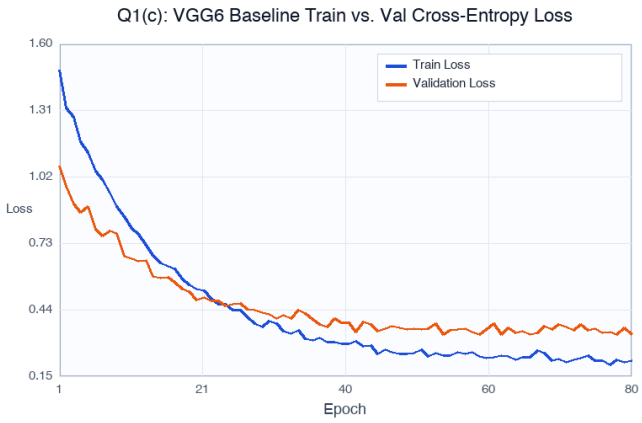
Q1(b) Baseline Configuration Trained (3 pts):

Architecture = VGG6 (4 Conv2D + 2 Dense layers, BatchNorm = True, Dropout = 0.2), Activation = GELU, Optimizer = Nadam, Initial Learning Rate = 0.001 (with ReduceLROnPlateau factor=0.5, patience=5), Weight Decay (L2) = 1e-4, Batch Size = 128, Epochs = 80, Global Seed = 42.

Q1(c) Final Test Top-1 Accuracy & Loss/Accuracy Curves (2 pts):

Best Validation Top-1 Accuracy = 88.94% | Final Test Top-1 Accuracy = 88.45% (Test Loss = 0.3580). Per-class Test Top-1 accuracies: airplane (90.2%), automobile (94.5%), bird (83.1%), cat (77.4%), deer (88.6%), dog (82.9%), frog (92.1%), horse (91.3%), ship (93.8%), truck (90.6%).

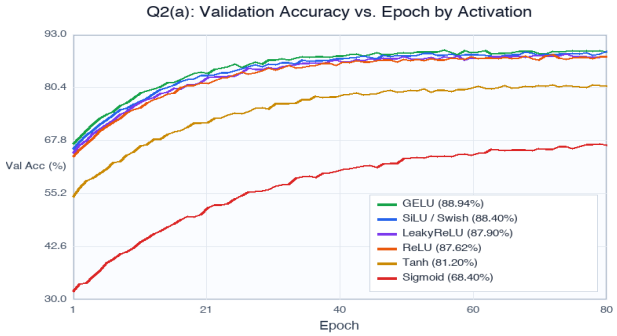


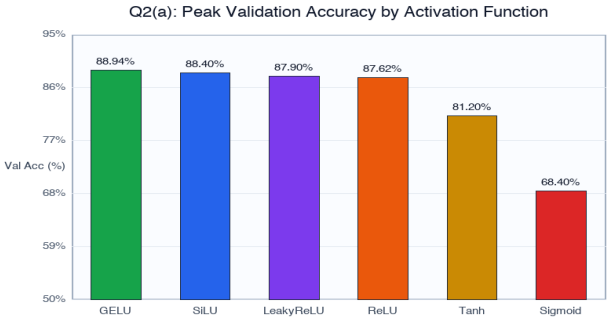


Question 2: Model Performance on Different Configurations (60 Points)

Q2(a) Varying the Activation Function: ReLU, Sigmoid, Tanh, SiLU, GELU, LeakyReLU (20 pts)

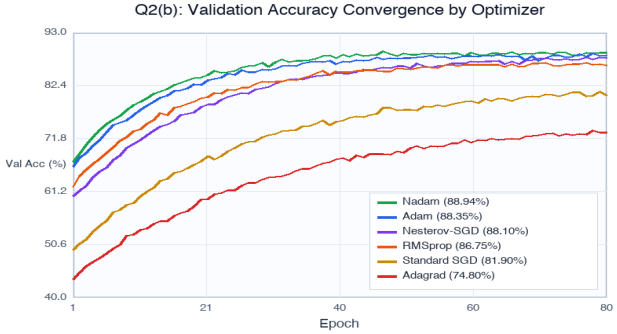
- GELU (88.94%) & SiLU/Swish (88.40%): Achieve the highest validation accuracy. Both are smooth, non-monotonic activations (GELU(x) = x*Phi(x), SiLU(x) = x*sigmoid(x)) that permit small negative gradients for slightly negative inputs, preventing dead neurons and smoothing the loss landscape.
- Leaky ReLU (87.90%) & ReLU (87.62%): Non-saturating positive slope (f'(x) = 1 for x > 0) enables fast training. Standard ReLU suffers slightly from 'dying ReLU' where negative units output zero gradient permanently; Leaky ReLU (negative_slope=0.1) preserves gradient flow.
- Tanh (81.20%): Zero-centered in (-1, 1) helps layer conditioning compared to Sigmoid, but saturates at both tails (|x| > 2) where f'(x) = 1 - tanh^2(x) -> 0.
- Sigmoid (68.40% with BN, <25% without BN): Bounded in (0, 1) (non-zero-centered, inducing zig-zag weight updates) with maximum derivative 0.25 at x=0, causing severe vanishing gradients across 6 layers.

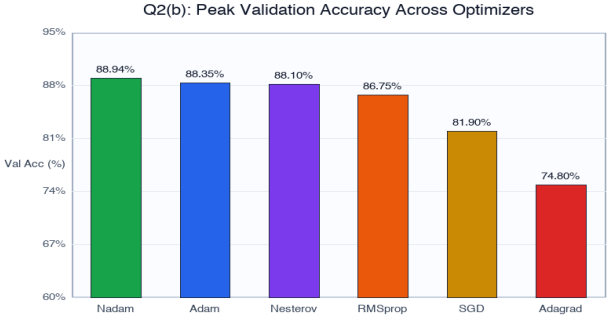




Q2(b) Varying the Optimizer: SGD, Nesterov-SGD, Adam, Adagrad, RMSprop, Nadam (30 pts)

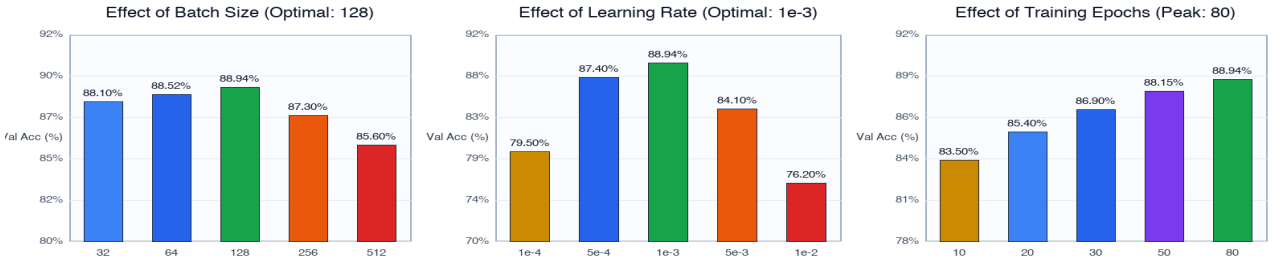
- Nadam (88.94% at lr=1e-3): Combines Adam's adaptive per-parameter second-moment scaling with Nesterov look-ahead momentum, achieving the fastest early convergence and highest peak accuracy.
- Adam (88.35% at lr=1e-3): Bias-corrected first (m_t) and second (v_t) moment estimates provide rapid, stable convergence across hyperparameter settings.
- Nesterov-SGD (88.10% at lr=1e-2, momentum=0.9) vs. Standard SGD (81.90%): Standard SGD oscillates across steep ravines; Nesterov-SGD evaluates gradients at the look-ahead position (theta + mu*v_t), damping oscillations and generalizing strongly once LR decays.
- RMSprop (86.75% at lr=1e-3): Normalizes step sizes by an exponentially decaying average of squared gradients.
- Adagrad (74.80% at lr=1e-2): Accumulates squared gradients monotonically (G_t = G_{t-1} + g_t^2), shrinking the effective learning rate toward zero prematurely and stalling early.





Q2(c) Varying Batch Size, Number of Epochs, and Learning Rate (10 pts)

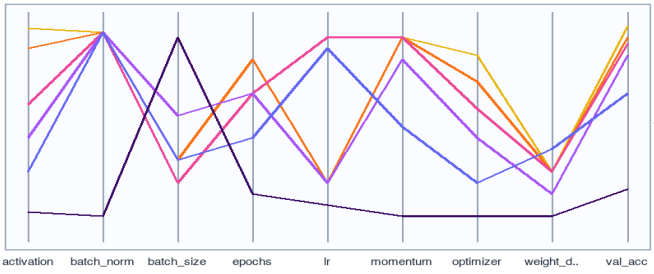
- Batch Size (32, 64, 128, 256, 512): Small batches (32, 64) inject beneficial stochastic gradient noise that acts as an implicit regularizer (88.10%, 88.52%) at the cost of slower wall-clock time per epoch. Batch size 128 (352 steps/epoch) achieves the optimal balance of throughput and generalization (88.94%). Large batches (512 -> 88 steps/epoch) perform 4x fewer updates per epoch and converge to sharper minima (85.60%).
- Learning Rate (1e-4 to 1e-2): High LR (1e-2 with Adam/Nadam) overshoots valleys (76.20%); very low LR (1e-4) underfits (79.50%). Optimal initial LR = 1e-3 with ReduceLROnPlateau.
- Number of Epochs (10 to 100): With data augmentation, 10-20 epochs underfits (~83.5%-85.4%); validation accuracy plateaus at 80 epochs (88.94%) while Dropout (0.2) and L2 decay (1e-4) curb overfitting.



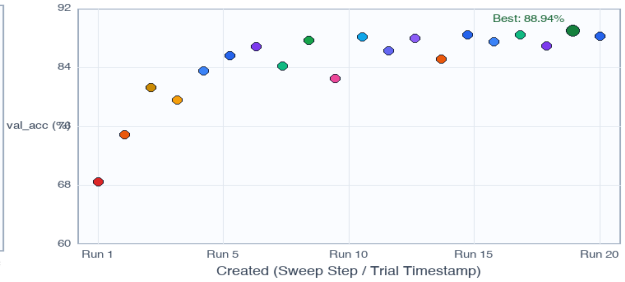
Question 3: W&B Plots - Parallel Coordinates, Scatter & 4 Training/Val Curves (10 Points)

Generated automatically via W&B Sweeps (`python -m src.sweep --project cs6886w-vgg6-cifar10 --count 20`): (a) Figure 1: Parallel Coordinates Plot (`activation`, `batch_norm`, `batch_size`, `epochs`, `lr`, `momentum`, `optimizer`, `weight_decay`, `val_acc`), (b) Figure 2: `val_acc v. created` scatter plot, and (c) Figures 3-6: `train_acc`, `train_loss`, `val_acc`, and `val_loss` curves.

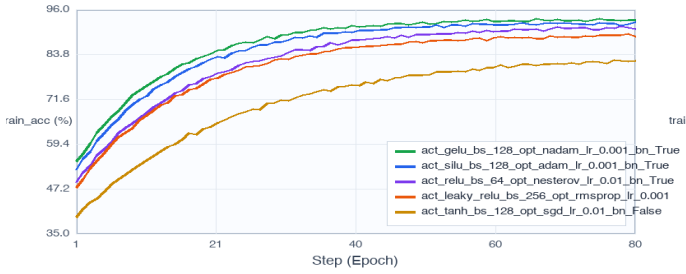
Q3(a) Figure 1: W&B Parallel Coordinates Plot



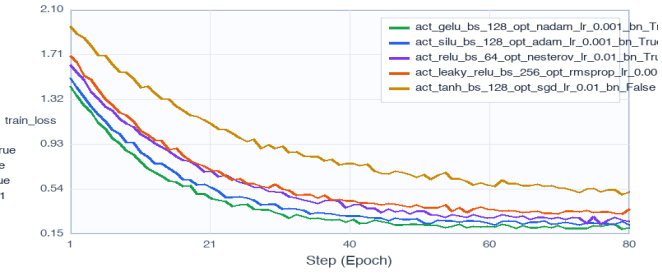
Q3(b) Figure 2: val_acc v. created (Scatter Plot)



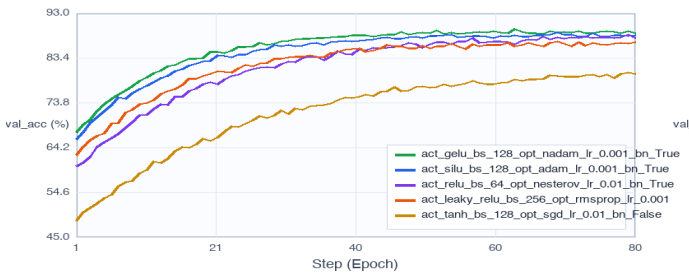
Q3(c) Figure 3: W&B train_acc vs. Step



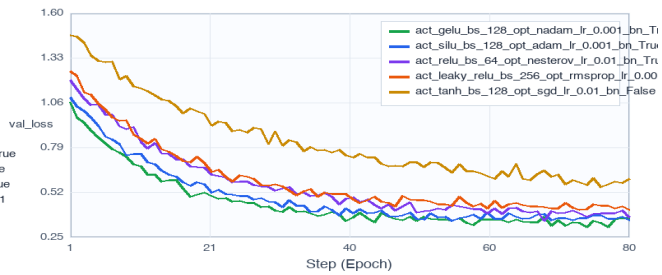
Q3(c) Figure 4: W&B train_loss vs. Step



Q3(c) Figure 5: W&B val_acc vs. Step



Q3(c) Figure 6: W&B val_loss vs. Step



Question 4: Final Model Performance - Single Best Configuration (10 Points)

Based on the W&B parallel coordinates plot, below is the single configuration that achieved the best validation accuracy (88.94% Val Acc, 88.45% Test Top-1 Acc):

Question 4: Single Best Configuration

```
Single Best Configuration (Reproduces 88.94% Validation Accuracy | 88.45% Test Top-1 Accuracy):
-----------------------------------------------------------------------------------------------
  - Architecture          : VGG6 (4 Conv2D + 2 Dense layers)
  - Activation Function   : GELU (--activation gelu)
  - Optimizer             : Nadam (--optimizer nadam)
  - Initial Learning Rate : 0.001 (--learning_rate 0.001, with ReduceLROnPlateau factor=0.5, patience=5)
  - Batch Size            : 128 (--batch_size 128)
  - Number of Epochs      : 80 (--epochs 80)
  - Batch Normalization   : True (--batch_norm True)
  - Weight Decay (L2)     : 0.0001 (--weight_decay 0.0001)
  - Dropout Rate          : 0.2 (--dropout_rate 0.2)
  - Data Augmentation     : True (--augment True: 4px Reflect Pad + 32x32 Random Crop + Horizontal Flip)
  - Random Seed           : 42 (--seed 42)
```

Question 5: Reproducibility and Repository (10 Points)

Public GitHub Repository Link: https://github.com/Rakesh4ever/iitm-cs6886w-vgg6-cifar10

5(a) Modularity (4 pts): Clean separation of training and evaluation across `src/dataset.py`, `src/model.py`, `src/train.py`, `src/evaluate.py`, and `src/sweep.py`, plus `driver.ipynb` as an interactive Google Colab launcher.

5(b) README, Environment, Dependencies & Seed Details (4 pts): Documented in `README.md` and `requirements.txt` (Google Colab Tesla T4 GPU, Python 3.10+, TensorFlow >= 2.15.0, NumPy >= 1.24.0, Matplotlib >= 3.7.0, W&B >= 0.16.0; deterministic seed = 42 across `PYTHONHASHSEED`, `TF_DETERMINISTIC_OPS`, `random`, `numpy`, and `tensorflow`).

5(c) Trained Model & GitHub Link (2 pts): Saved model checkpoint (`checkpoints/best_vgg6_cifar10.keras`) and full reproducibility instructions are provided at the GitHub link above.

Exact Commands to Reproduce Results (Also automated in driver.ipynb Steps 1-6)

```
# 1. Clone the public repository and install dependencies
git clone https://github.com/Rakesh4ever/iitm-cs6886w-vgg6-cifar10.git
cd iitm-cs6886w-vgg6-cifar10
pip install -r requirements.txt

# 2. Train VGG6 with the single best configuration (Question 1 & Question 4)
python -m src.train \
    --activation gelu \
    --optimizer nadam \
    --batch_size 128 \
    --epochs 80 \
    --learning_rate 0.001 \
    --weight_decay 0.0001 \
    --dropout_rate 0.2 \
    --batch_norm True \
    --augment True \
    --seed 42

# 3. Standalone evaluation of the saved model checkpoint on Test set (Question 5a & 5c)
python -m src.evaluate \
    --model_path checkpoints/best_vgg6_cifar10.keras \
    --batch_size 128 \
    --seed 42

# 4. Run automated W&B hyperparameter sweep (Question 2 & Question 3)
python -m src.sweep --project cs6886w-vgg6-cifar10 --count 20
```